Experiment No.6
Implementation of Recursive Decent Parser (RDP)
Name: Harsh Nalawade
Roll no. 38
Date of Performance:21/1/26
Date of Submission:28/1/26

Aim: Implementation of Recursive Decent Parser (RDP).

Objective: To develop an ability to design and develop the Analysis phase of Compiler.

Theory:

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right. It uses procedures for every terminal and non-terminal entity. This parsing technique recursively parses the input to make a parse tree, which may or may not require back-tracking. But the grammar associated with it (if not left factored) cannot avoid back-tracking. A form of recursive-descent parsing that does not require any backtracking is known as predictive parsing. This parsing technique is regarded recursive as it uses context-free grammar which is recursive in nature.

Back-tracking

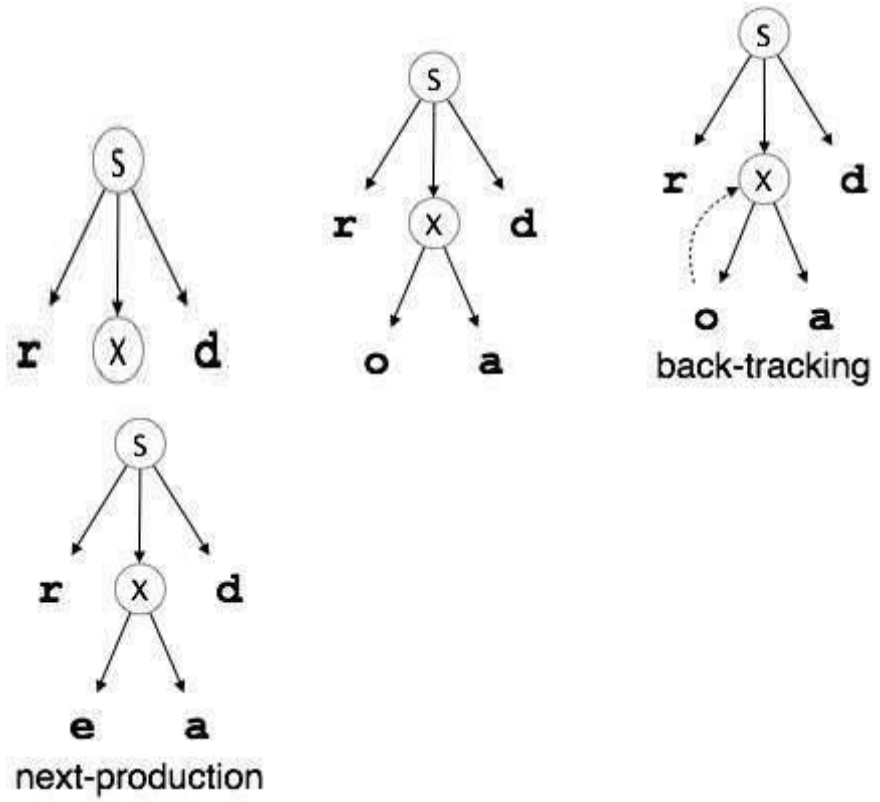
Top- down parsers start from the root node (start symbol) and match the input string against the production rules to replace them (if matched). To understand this, take the following example of CFG:

S → rXd | rZd
X → oa | ea
Z → ai

For an input string: read, a top-down parser, will behave like this:

It will start with S from the production rules and will match its yield to the left-most letter of the input, i.e. 'r'. The very production of S (S → rXd) matches with it. So the top-down parser advances to the next input letter (i.e. 'e'). The parser tries to expand non-terminal 'X' and checks its production from the left (X → oa). It does not match with the next input symbol. So the top-down parser backtracks to obtain the next production rule of X, (X → ea).

Now the parser matches all the input letters in an ordered manner. The string is accepted.



Code:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdlib.h>

// Enhanced Structure for a Variable/Symbol Table
struct SymbolTable {
    char name[31];
    char type[20];      // Data type (int, float, etc.)
    char category[20];  // Role (Variable, Constant, etc.)
} st[100];
```

```
int st_count = 0;
char current_declared_type[20] = "Identifier"; // Tracks the last seen keyword type

int isKeyword(char buffer[]) {
    char keywords[32][10] = {"auto","break","case","char","const","continue","default",
                             "do","double","else","enum","extern","float","for","goto",
                             "if","int","long","register","return","short","signed",
                             "sizeof","static","struct","switch","typedef","union",
                             "unsigned","void","volatile","while"};
    for(int i = 0; i < 32; ++i) {
        if(strcmp(keywords[i], buffer) == 0) return 1;
    }
    return 0;
}

// Adds to table with type awareness and duplicate checking
void addToSymbolTable(char name[], char type[]) {
    for(int i = 0; i < st_count; i++) {
        if(strcmp(st[i].name, name) == 0) return; // Already exists
    }
    strcpy(st[st_count].name, name);
    strcpy(st[st_count].type, type);
    strcpy(st[st_count].category, "Variable");
    st_count++;
}
```

```
int main() {
    char ch, buffer[31], operators[] = "+-*/%=";
    char expression[100];
    int i, j = 0;

    printf("Enter a C statement (e.g., 'int count = 10;'): ");
    fgets(expression, sizeof(expression), stdin);

    printf("\n--- Token Analysis ---\n");

    for(i = 0; i < strlen(expression); ++i) {
        ch = expression[i];

        // 1. Identify Operators
        for(int k = 0; k < 6; ++k) {
            if(ch == operators[k]) {
                printf("Token: %c\t\tType: Operator\n", ch);
            }
        }

        // 2. Handle Alphanumeric Buffers (Keywords, Identifiers, Numbers)
        if(isalnum(ch)) {
            buffer[j++] = ch;
        }
        else if((isspace(ch) || ispunct(ch)) && (j != 0)) {
```

```
        buffer[j] = '\0';
        j = 0;

        if(isKeyword(buffer)) {
            printf("Token: %s\t\tType: Keyword\n", buffer);
            // If it's a type keyword, save it for the next identifier
            if(strcmp(buffer, "int") == 0 || strcmp(buffer, "float") == 0 ||
               strcmp(buffer, "char") == 0 || strcmp(buffer, "double") == 0) {
                strcpy(current_declared_type, buffer);
            }
        }
        else if(isdigit(buffer[0])) {
            printf("Token: %s\t\tType: Constant\n", buffer);
        }
        else {
            printf("Token: %s\t\tType: Identifier\n", buffer);
            addToSymbolTable(buffer, current_declared_type);
        }
    }

    // 3. Punctuation
    if(ch == ';' || ch == '(' || ch == ')' || ch == ',') {
        printf("Token: %c\t\tType: Punctuation\n", ch);
        // Reset type after a semicolon (end of statement)
        if(ch == ';') strcpy(current_declared_type, "Identifier");
    }
```

```
    }

    // Display Final Variable Table
    printf("\n--- FINAL SYMBOL / VARIABLE TABLE ---\n");
    printf("--------------------------------------------\n");
    printf("Index\tName\t\tData Type\tCategory\n");
    printf("--------------------------------------------\n");
    for(i = 0; i < st_count; i++) {
        printf("%d\t%-10s\t%-10s\t%s\n", i + 1, st[i].name, st[i].type, st[i].category);
    }
    printf("--------------------------------------------\n");

    return 0;
}
```

Output:

```
Enter a C statement (e.g., 'int count = 10;'): int count =10;

--- Token Analysis ---
Token: int      Type: Keyword
Token: count        Type: Identifier
Token: =        Type: Operator
Token: 10       Type: Constant
Token: ;        Type: Punctuation

--- FINAL SYMBOL / VARIABLE TABLE ---s
-------------------------------------------
Index   Name        Data Type   Category
-------------------------------------------
1   count       int         Variable
-------------------------------------------
```

Conclusion:

The Recursive Descent Parser reads an input string and checks it against a given grammar using recursive functions for each non-terminal. It successfully identifies whether a string is **accepted** or **rejected** based on the grammar rules. The program demonstrates how syntactic analysis works in a compiler by validating the structure of the input according to the language's grammar.